

Collections Framework

List, Set, Map, Queue; Comparable & Comparator; iteration patterns;
when to use HashMap vs TreeMap, ArrayList vs LinkedList

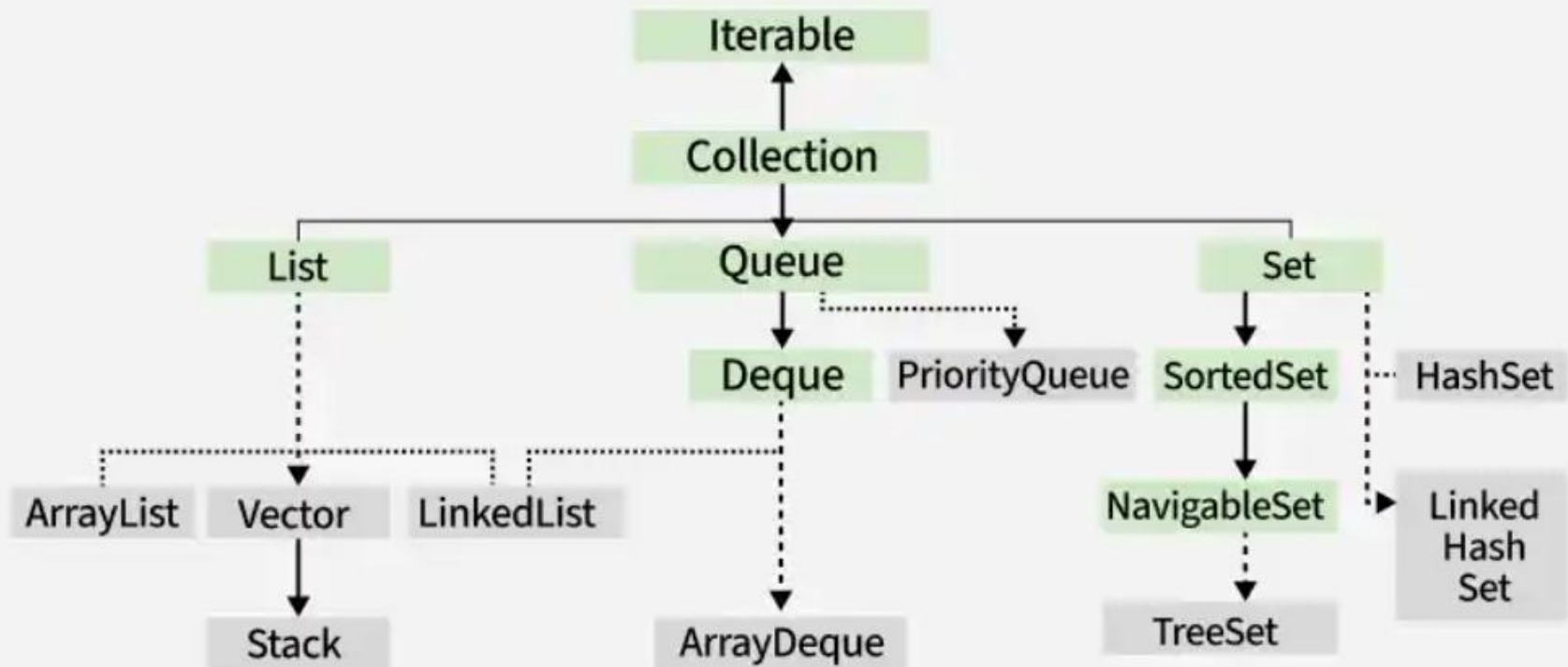
Rahul Guha || SME || Java FS Training || HCL

Java Collection Framework

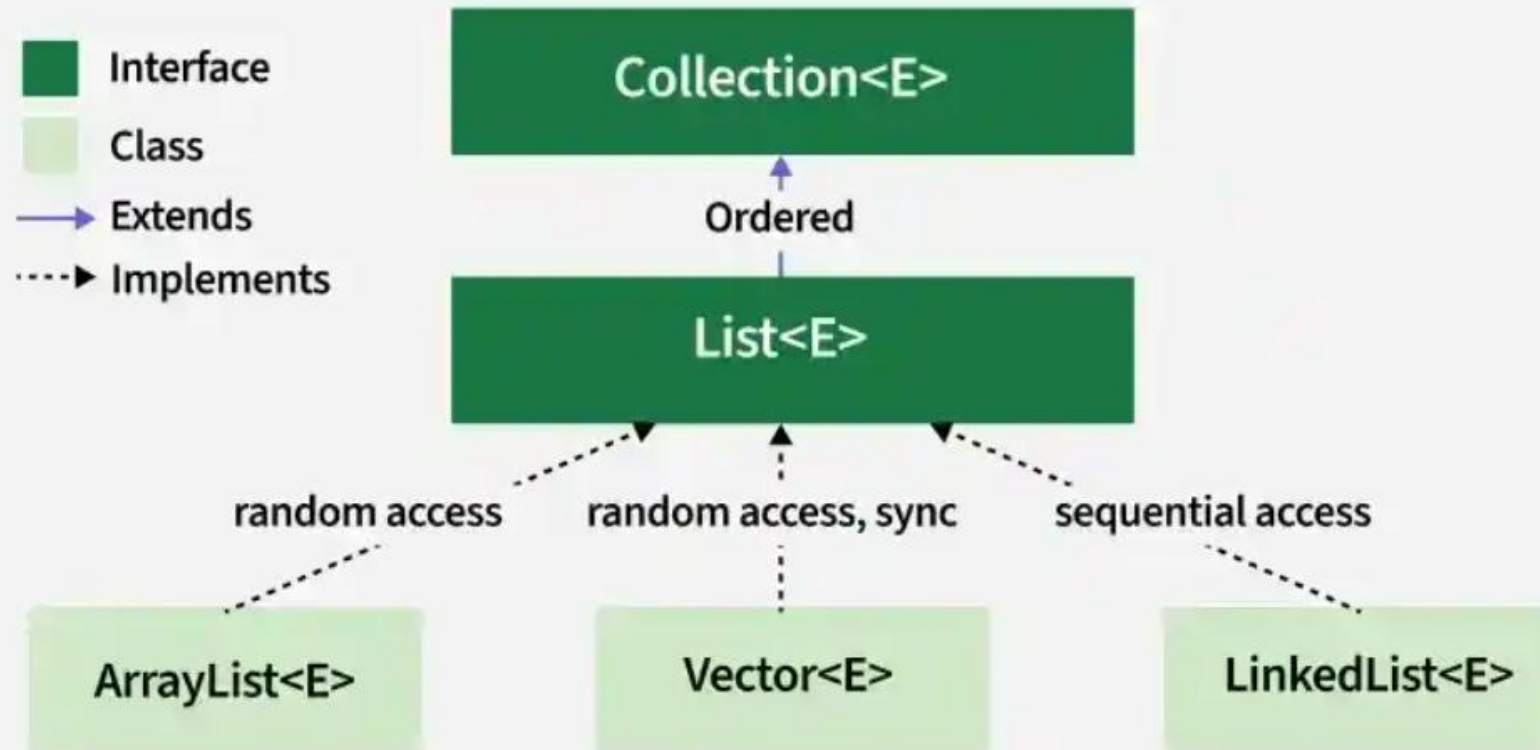
Java Collection Framework (JCF) is a set of classes and interfaces that provide ready-made data structures to store and manipulate groups of objects efficiently.

- Java provides collection interfaces like List, Set, Map, and Queue, with ready-made classes such as ArrayList, HashSet, HashMap, and PriorityQueue, so you don't have to write data-handling code from scratch.
- The Collection Framework improves productivity by making code more reusable, maintainable and faster to develop.
- All of these are part of the java.util package.
- They are used to store, search, sort, and organize data more easily - all using standardized methods and patterns.

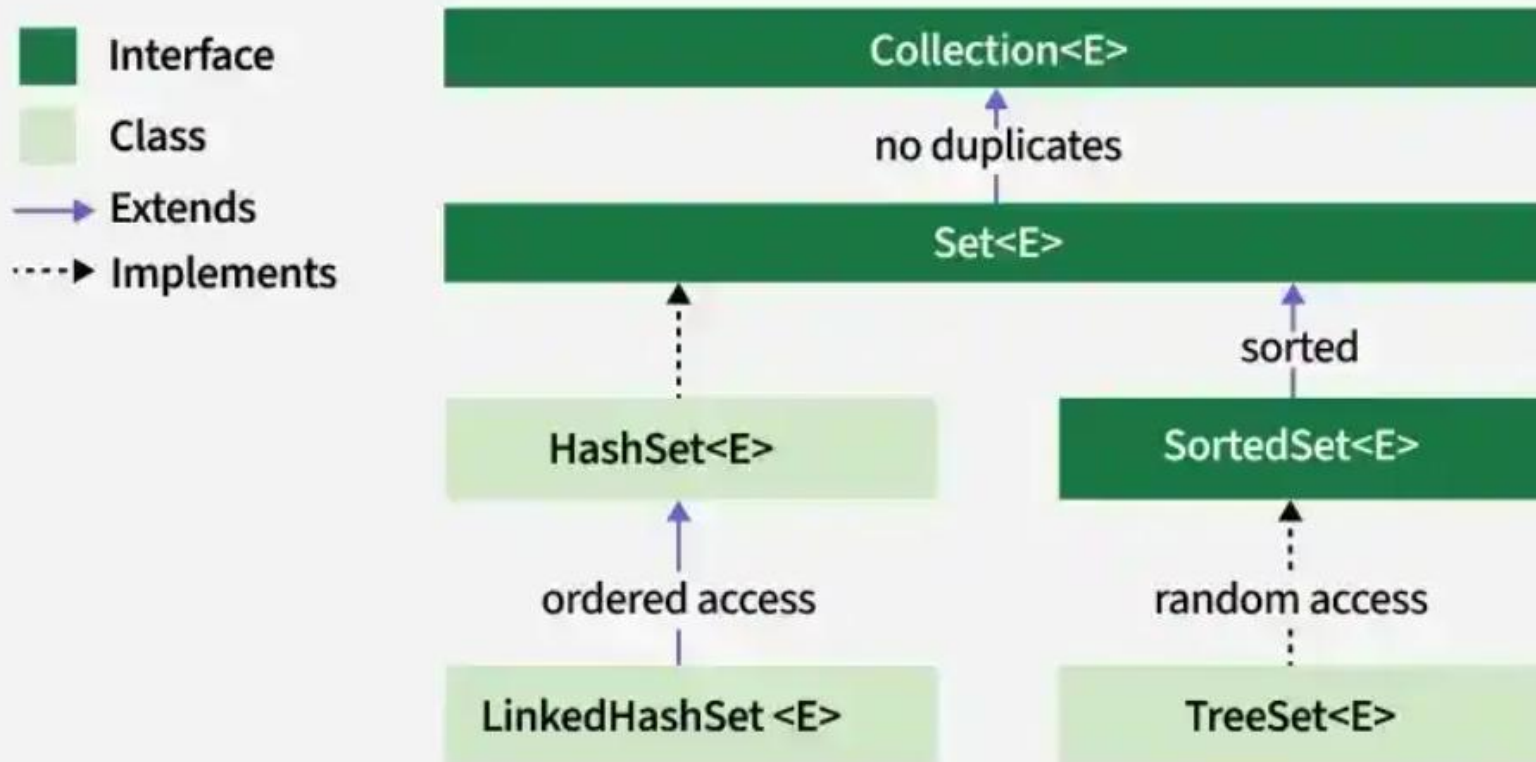
Java Collection Framework



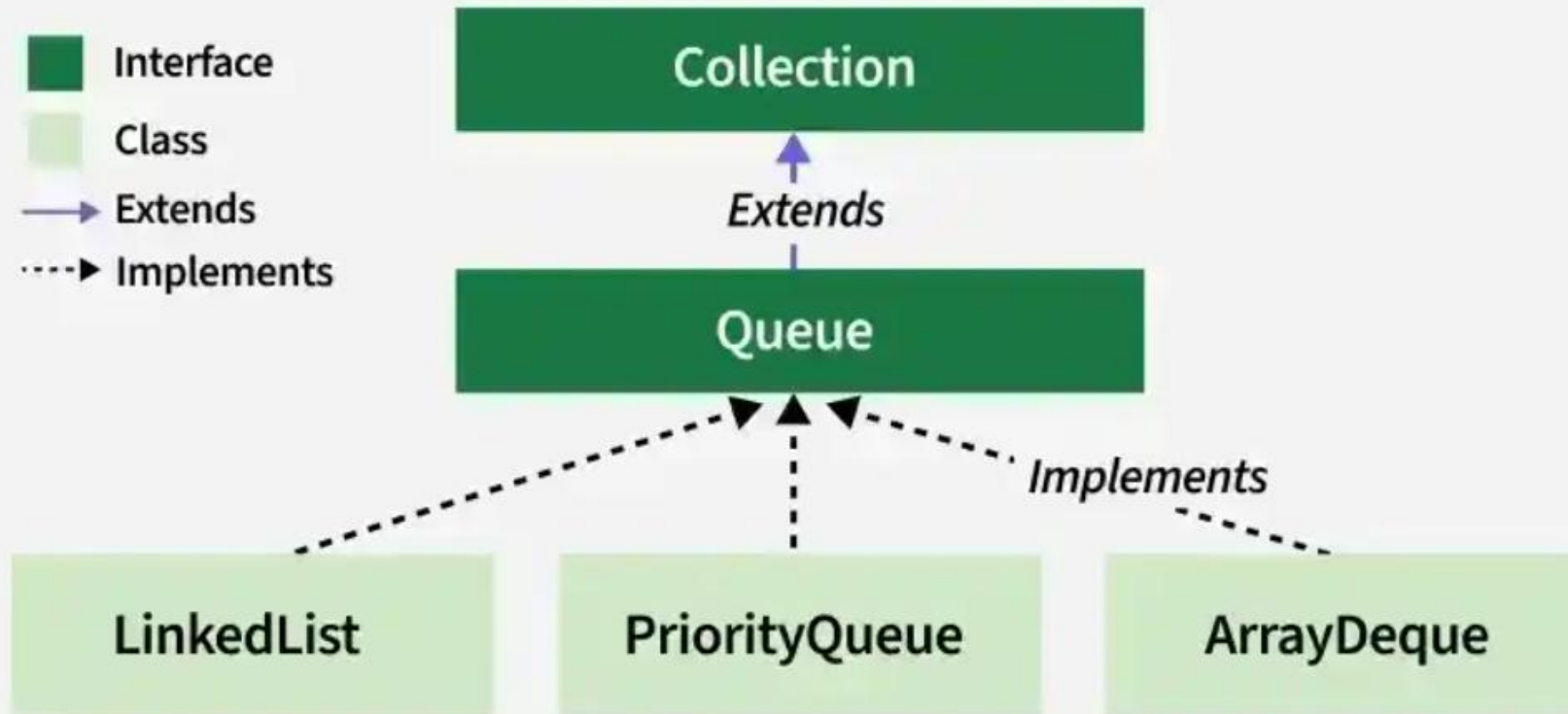
Java Collection Framework



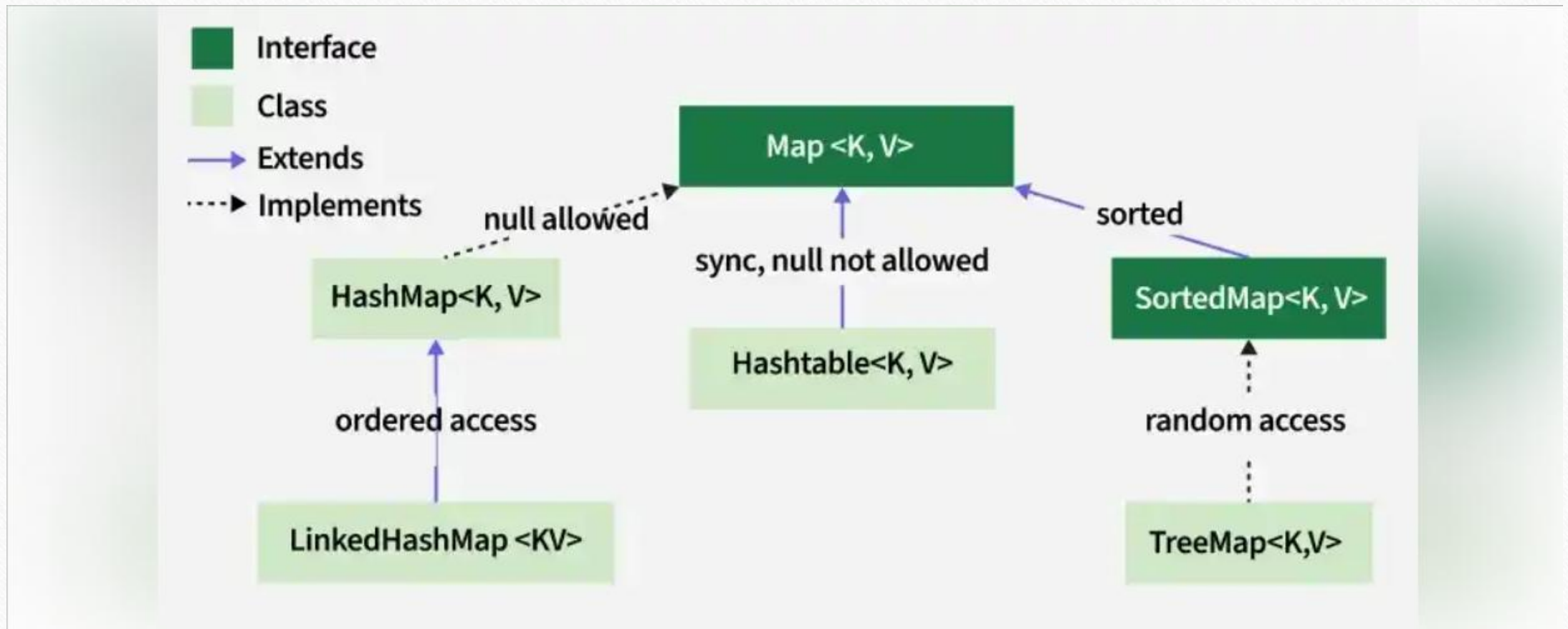
Java Collection Framework



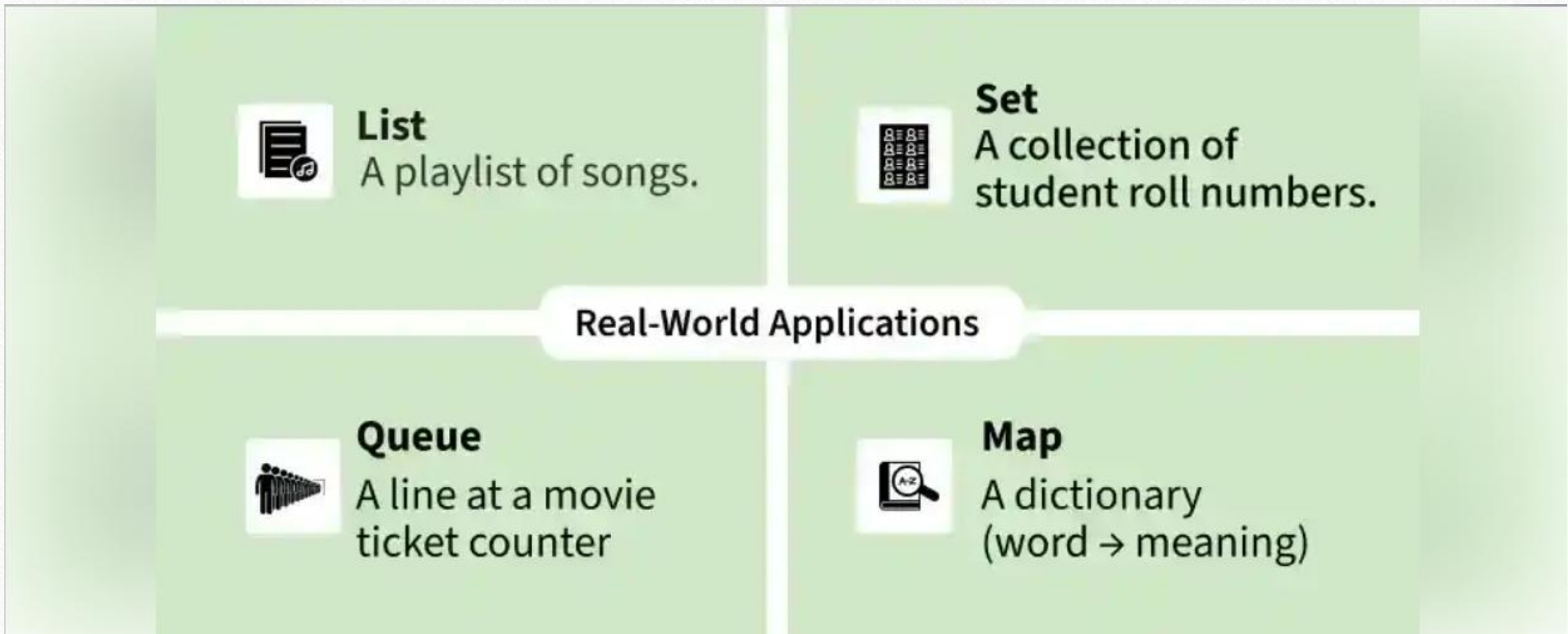
Java Collection Framework



Java Collection Framework

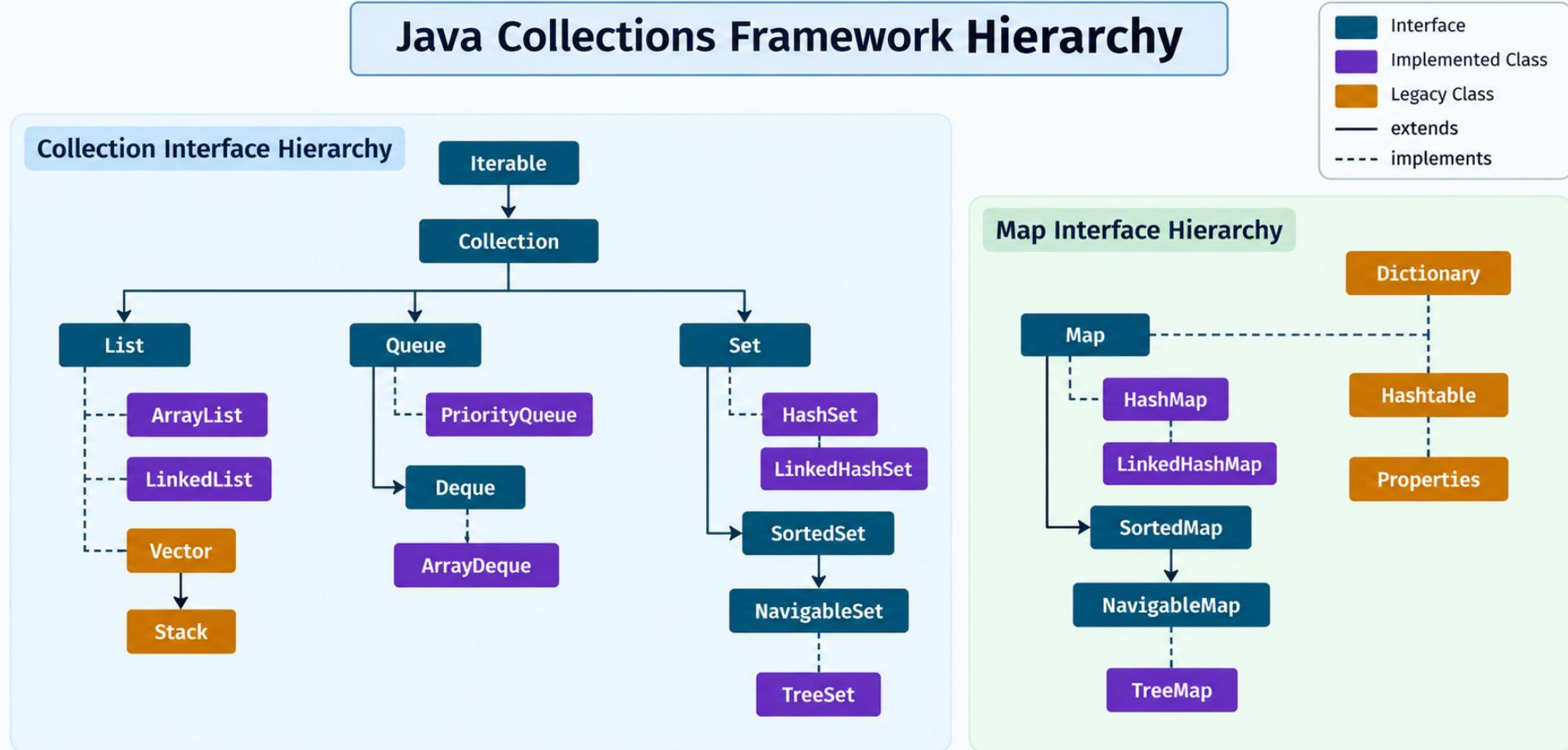


Java Collection Framework



Java Collection Framework

Java Collections Framework Hierarchy



Java Collection Framework

Hierarchy of Collections Framework

The Collections Framework in Java contains two main interfaces:

- Collection – Represents a group of objects, like List, Set, or Queue.
- Map – Represents a collection of key-value pairs.
- Note: Map does not extend Collection.

Tip: Some classes like Vector, Stack, Hashtable, Properties, and Dictionary are considered legacy classes but are still part of the framework for backward compatibility.

The two main packages present in Collections Framework are:-

- java.util.Collection package
- java.util.Map package

Uses of Collections Framework

Storing Data:

Use ArrayList or LinkedList to store lists of students, products, or items.

Processing Queues:

Use Queue or Deque for tasks like managing print jobs or task scheduling.

Managing Unique Data:

Use HashSet or TreeSet to store unique elements like employee IDs.

Key-Value Mapping:

Use HashMap or TreeMap to store data in key-value pairs like country-capital mapping.

Sorting and Searching:

Collections classes provide built-in methods for sorting, searching, and shuffling data easily.

Advantages of Collections Framework

Reusability:

You can use predefined classes like ArrayList, HashMap, etc., without writing code from scratch.

Efficiency:

Collections classes are optimized for storing, retrieving, and manipulating data quickly.

Consistency:

Provides a standard way to work with data structures, so code is easier to read and maintain.

Flexibility:

Supports different types of collections like List, Set, Map for various needs.

Thread-Safety (optional):

Some collections like Vector and Hashtable are synchronized and safe for multithreaded use.

1. List

A List is an ordered collection that:

- Maintains insertion order
- Allows duplicate elements
- Allows index-based access
- Can contain multiple null values depending on implementation

Class	Ordering	Duplicates	Random Access	Typical Use
ArrayList	Insertion order	Yes	Fast	General-purpose list
LinkedList	Insertion order	Yes	Slow	Frequent insertion/deletion
Vector	Insertion order	Yes	Fast	Legacy synchronized list

2. LinkedList

LinkedList uses a doubly linked list internally. Each node contains: Previous | Data | Next. In Java, a LinkedList is a built-in class found in the java.util package that implements a doubly-linked list data structure. Unlike a standard array or ArrayList, it does not store elements in contiguous memory locations. Instead, each element resides in a separate container called a node, where each node holds the actual data and two references (pointers): one pointing to the next node and one pointing to the previous node.

Key Characteristics

- **Dynamic Size:** It automatically grows and shrinks as elements are added or removed.
- **Maintains Order:** Elements preserve their insertion order.
- **Duplicates & Nulls Allowed:** You can store duplicate items as well as null values.
- **Dual Interfaces:** It implements both the List and Deque interfaces, meaning it can function as a standard list, a queue, or a stack.
- **Not Thread-Safe:** By default, LinkedList is non-synchronized.

LinkedList Vs ArrayList

Feature	ArrayList	LinkedList
Internal structure	Dynamic array	Doubly linked list
Index access	Fast	Slow
get(index)	$O(1)$	$O(n)$
Add at end	Usually $O(1)$ amortized	$O(1)$
Insert/remove at beginning	$O(n)$	$O(1)$
Memory usage	Lower	Higher
Cache locality	Better	Poorer
General-purpose use	Excellent	Less common
Best for	Frequent reading/access	Frequent insertion/removal at ends

Set Comparison

Feature	HashSet	LinkedHashSet	TreeSet
Duplicates	No	No	No
Ordering	No guaranteed order	Insertion order	Sorted order
Performance	Generally fastest	Slight overhead	Generally slower
Internal structure	Hash table	Hash table + linked structure	Tree
Sorting	No	No	Yes
Use when	Fast membership checking	Need insertion order	Need sorted unique data

Important HashMap Methods

put(key, value)
get(key)
remove(key)
containsKey(key)
containsValue(value)
size()
isEmpty()
clear()
keySet()
values()
entrySet()

Feature	HashMap	TreeMap
Ordering	No guaranteed order	Sorted by key
Internal structure	Hash table	Red-Black tree
Average basic lookup	O(1)	O(log n)
Sorting	No	Yes
Null key	Allows one null key	Generally does not allow null key with natural ordering
Use case	Fast lookup	Sorted key-value data

Queue Methods: add vs offer, remove vs poll

Operation	Method	If operation fails
Insert	add()	Throws exception
Insert	offer()	Returns false
Examine	element()	Throws exception
Examine	peek()	Returns null
Remove	remove()	Throws exception
Remove	poll()	Returns null

Comparable vs Comparator

Feature	Comparable	Comparator
Package	java.lang	java.util
Main method	compareTo()	compare()
Location of logic	Inside class	Outside class
Sorting type	Natural ordering	Custom ordering
Number of orderings	Usually one	Multiple possible
Modifies original class	Yes	No
Example	Sort Student by ID	Sort Student by name/marks